Aaron N. Cutshall, DHA, MSHI

# How to Migrate from SQL Server to Postgres



SQLRV, LLC

HEALTHCARE INFORMATICS

https://www.sqlrv.com



# Just who is this guy?

SQLRV, LLC
HEALTHCARE INFORMATICS

Principal Data Architect, Population Health

BYU PATHWAY WORLDWIDE

Adjunct Instructor

redgate AMBASSADOR

Ambassador Program

DGIQ + EDW DATA SUMMIT
UNLEASH THE POWER OF YOUR DATA

Speaker – various events

B.S. Computer Science

M.S. Computer Information Systems

M.S. Health Informatics

VIRGINIA UNIVERSITY OF LYNCHBURG · SIBI AUXILIUM · ET LIBERTAS · 1886

Doctor of Healthcare Administration

40 Years



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Agenda

- Why Migrate from SQL Server to Postgres?
- Data Storage
  - Schemas
  - Tables
  - Data Types
  - Custom Data Types
  - Data Load
  - Primary Keys
  - Indexes
  - Foreign Keys



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Agenda

- Programming
  - Views
  - Stored Procedures
  - Functions
  - Triggers
- Bonus (if we have time)
  - Database Change Tracking
  - Database Error Logging
  - Other Housekeeping



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Something to consider…



"91% of teams – no matter how experienced – still hit at least one significant challenge during their [database migration]. The culprit usually isn't poor execution, but the things no one thought to plan for…the 'unknown unknowns.'"

– Grant Fritchey *(Data Platform MVP, Author, & Redgate Advocate)*

Copyright © 2026, SQLRV, LLC - All Rights Reserved



# Data Storage

- Case Sensitivity ** NOTE WELL **
  - SQL Server is case *in*sensitive ('Bob' = 'BOB' = 'bob')
  - Postgres is case sensitive ('Bob' ≠ 'BOB' ≠ 'bob')
  - Objects stored in lowercase unless quoted
- Schemas
  - More pronounced usage in Postgres than SQL Server
  - Provides more organizational capabilities
  - Some folks use them like mini-databases
- Tables
  - Conceptually similar between Postgres and SQL Server
  - Quote mechanism is different – double quotes vs square brackets



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Data Storage

- Data Types
  - Many are identical (date, int, decimal, etc.)
  - Some have different names with some minor differences
    - NCHAR / NVARCHAR vs char / varchar
    - DATETIME vs timestamp
    - BIT vs boolean
    - GUID vs uuid
  - Some seem to be the same but are implemented differently
    - GEOGRAPHY vs geography
    - HIERARCHYID vs ltree



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Data Storage

| Data Category | Data Type | SQL Server | PostgreSQL |
|---|---|---|---|
| **Binary Data** | Variable length byte string, 1 <= n <= 8000 | VARBINARY(n) | bytea |
| **Binary Data** | Universally (Globally) Unique Identifier | GUID | uuid |
| **Boolean** | 1, 0 or NULL *(Postgres allows True, False, Yes, No, etc.)* | BIT | boolean (bool) |
| **Character** | Fixed length char string, 1 <= n <= 8000 | CHAR(n) | char(n) |
| **Character** | Variable length char string, 1 <= n <= 8000 | VARCHAR(n) | varchar(n) |
| **Character** | Variable length Unicode UCS-2 data, <= 2GB | NVARCHAR(max) | text |
| **Character** | Variable length Unicode UCS-2 string | NVARCHAR(n) | varchar(n) |
| **Date/Time** | Date and time with fractional seconds | DATETIME | timestamp(p) |
| **Date/Time** | Date includes year, month, and day | DATE | date |
| **Geography** | Used to reference longitude and latitude | GEOGRAPHY | geography*+ |
| **Hierarchical** | Representation of hierarchical structures | HIERARCHYID | ltree*+ |
| **Numeric** | 8-bit integer (unsigned), 0 to 255 | TINYINT | smallint (int2)* |
| **Numeric** | 16-bit integer | SMALLINT | smallint (int2) |
| **Numeric** | 32-bit integer | INTEGER | integer (int, int4) |
| **Numeric** | 64-bit integer | BIGINT | bigint (int8) |
| **Numeric** | Currency amount (32-bit) | SMALLMONEY | numeric(8,4)* |
| **Numeric** | Currency amount (64-bit) | MONEY | numeric(10,4)* |

*Not an exact equivalent  +Extension

Copyright © 2026, SQLRV, LLC - All Rights Reserved



# Data Storage

- Data Types (cont.)
  - Postgres extensions provide additional capabilities
    - Data types and other functionality
    - Somewhat like CLR in SQL Server but more seamlessly integrated
  - Examples:

```
-- Create the extension to handle GUID/UUID values
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
-- Create the extension to handle hash values
CREATE EXTENSION IF NOT EXISTS pgcrypto;
-- Create the extension to handle heirarchical trees
CREATE EXTENSION IF NOT EXISTS ltree;
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Data Storage

- Data Types (cont.)
  - Some types do not exist so substitutions must be made

    Ex: `MONEY` → `numeric(20,4)` (money not recommended)
  - Identity columns are treated a little differently

    `IDENTITY(1,1)` → `GENERATED BY DEFAULT AS IDENTITY`
  - Defaults in Postgres are not constraints (no name needed)
  - Postgres has one type (`bytea`) for all binary data
  - Postgres supports XML data but not schema (XSD) validation
  - Postgres supports JSON extensively with great functionality



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Data Storage

- Custom Data Types
  - SQL Server:

    Format: `CREATE TYPE TypeName FROM DataType NullConstraint;`

    Example: `CREATE TYPE [Flag] FROM BIT NOT NULL;`
  - Postgres:

    Format: `CREATE DOMAIN DomainName FROM DataType NullConstraint;`

    Example: `CREATE DOMAIN d_flag FROM boolean NOT NULL;`



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Data Storage

```
-- SQL Server
CREATE TABLE [Person].[Address](
    [AddressID] [int] IDENTITY (1, 1) NOT FOR REPLICATION NOT NULL,
    [AddressLine1] [nvarchar](60) NOT NULL,
    [AddressLine2] [nvarchar](60) NULL,
    [City] [nvarchar](30) NOT NULL,
    [StateProvinceID] [int] NOT NULL,
    [PostalCode] [nvarchar](15) NOT NULL,
    [SpatialLocation] [geography] NULL,
    [rowguid] uniqueidentifier ROWGUIDCOL NOT NULL CONSTRAINT [DF_Address_rowguid] DEFAULT (NEWID()),
    [ModifiedDate] [datetime] NOT NULL CONSTRAINT [DF_Address_ModifiedDate] DEFAULT (GETDATE())
) ON [PRIMARY];

-- Postgres
CREATE TABLE Person.Address (
     AddressID INT GENERATED BY DEFAULT AS IDENTITY,
     AddressLine1 varchar(60) NOT NULL,
     AddressLine2 varchar(60) NULL,
     City varchar(30) NOT NULL,
     StateProvinceID int NOT NULL,
     PostalCode varchar(15) NOT NULL,
     SpatialLocation geography NULL, -- Representing longitude and latitude
     rowguid uuid not null DEFAULT gen_random_uuid(), -- constraint name ignored as defaults are not constraints in Postgres
     ModifiedDate timestamp NOT NULL DEFAULT now()
     );
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Data Storage

- Populate Tables
  - SQL Server uses BULK INSERT with many options
    - Multiple configurations used for SQL Server
    - Great flexibility with multiple methods
  - Postgres uses COPY command
    - Mostly targets delimited data (CSV, pipe delimited, etc.)
    - While not as flexible, it covers most use cases and is very fast!
  - Some data conversions are necessary
    - GEOGRAPHY -> Well-Known Text -> geography
    - GUID -> text -> uuid



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Data Storage

- Primary Keys
  - Usually defined with column (if for a single column only):

    *column_name data_type* `PRIMARY KEY`
  - Can be part of table definition:

    ```
    PRIMARY KEY (column_1, column_2, …);
    ```
  - Added after table creation

    ```
    ALTER TABLE table_name
        ADD PRIMARY KEY (column_1, column_2, …);
    ```
    - Used when data is populated first
    - Index is created when the primary key is defined



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Data Storage

- Indexes
  - Clustered indexes not defined in Postgres
  - Default index type is B-Tree, just like SQL Server
  - Other index types supported:
    - HASH – 32-bit value for only equality comparisons
    - GiST – Supports multiple indexing strategies and methods
    - GIN – Inverted indexes for multiple custom methods
    - BRIN – Block Range Indexes can be made similar to clustering
  - Syntax very much like SQL Server



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Data Storage

- Foreign Keys
  - Essentially a reference to another table's Primary Key
  - Implemented very similar in Postgres as in SQL Server

```
-- SQL Server
ALTER TABLE [Production].[BillOfMaterials] ADD
    CONSTRAINT [FK_BillOfMaterials_Product_ProductAssemblyID]
        FOREIGN KEY ([ProductAssemblyID]) REFERENCES [Production].[Product]([ProductID]),
    CONSTRAINT [FK_BillOfMaterials_Product_ComponentID]
        FOREIGN KEY ([ComponentID]) REFERENCES [Production].[Product]([ProductID]),
    CONSTRAINT [FK_BillOfMaterials_UnitMeasure_UnitMeasureCode]
        FOREIGN KEY ([UnitMeasureCode]) REFERENCES [Production].[UnitMeasure]([UnitMeasureCode]);

-- Postgres
ALTER TABLE Production.BillOfMaterials ADD
    CONSTRAINT FK_BillOfMaterials_Product_ProductAssemblyID
        FOREIGN KEY (ProductAssemblyID) REFERENCES Production.Product(ProductID),
    CONSTRAINT FK_BillOfMaterials_Product_ComponentID
        FOREIGN KEY (ComponentID) REFERENCES Production.Product(ProductID),
    CONSTRAINT FK_BillOfMaterials_UnitMeasure_UnitMeasureCode
        FOREIGN KEY (UnitMeasureCode) REFERENCES Production.UnitMeasure(UnitMeasureCode);
```

Copyright © 2026, SQLRV, LLC - All Rights Reserved





# Questions, anyone??



Remember:

Migrating data is easy; the rest is hard!





Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Programming

- Views
  - Logical Views
    - Simple Definition: Stored Query
    - Query gets executed when the view is referenced
    - Relies upon indexes within referenced tables
  - Materialized Views
    - Simple Definition: Stored Query Result Set
    - Result set is stored in an internal table when created or refreshed
    - Can be indexed as any other table would be indexed



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Programming

- Example View:

```
-- SQL Server
CREATE VIEW [Production].[vProductAndDescription]
WITH SCHEMABINDING
AS
-- View (indexed or standard) to display products and product descriptions by language.
SELECT p.[ProductID]
      ,p.[Name]
      ,pm.[Name] AS [ProductModel]
      ,pmx.[CultureID]
      ,pd.[Description]
FROM [Production].[Product] p
INNER JOIN [Production].[ProductModel] pm
    ON p.[ProductModelID] = pm.[ProductModelID]
INNER JOIN [Production].[ProductModelProductDescriptionCulture] pmx
    ON pm.[ProductModelID] = pmx.[ProductModelID]
INNER JOIN [Production].[ProductDescription] pd
    ON pmx.[ProductDescriptionID] = pd.[ProductDescriptionID];
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Programming

- Example View:

```
-- Postgres:
CREATE VIEW Production.vProductAndDescription AS
    SELECT  p.ProductID,
            p.Name,
            pm.Name AS ProductModel,
            pmx.CultureID,
            pd.Description,
    FROM    Production.Product p
    INNER JOIN Production.ProductModel pm
        ON  pm.ProductModelID = p.ProductModelID
    INNER JOIN Production.ProductModelProductDescriptionCulture pmx
        ON  pmx.ProductModelID = pm.ProductModelID
    INNER JOIN Production.ProductDescription pd
        ON  pd.ProductDescriptionID = pmx.ProductDescriptionID;
```

  - Postgres views are *always* schema-bound

Copyright © 2026, SQLRV, LLC - All Rights Reserved



# Programming

- Example View:

```
-- SQL Server
CREATE VIEW [Production].[vProductAndDescription]
WITH SCHEMABINDING
AS
-- View (indexed or standard) to display products and product descriptions by language.
SELECT p.[ProductID]
      ,p.[Name]
      ,pm.[Name] AS [ProductModel]
      ,pmx.[CultureID]
      ,pd.[Description]
FROM [Production].[Product] p
INNER JOIN [Production].[ProductModel] pm
    ON p.[ProductModelID] = pm.[ProductModelID]
INNER JOIN [Production].[ProductModelProductDescriptionCulture] pmx
    ON pm.[ProductModelID] = pmx.[ProductModelID]
INNER JOIN [Production].[ProductDescription] pd
    ON pmx.[ProductDescriptionID] = pd.[ProductDescriptionID];
GO

-- Index the vProductAndDescription view
CREATE UNIQUE CLUSTERED INDEX [IX_vProductAndDescription] ON [Production].[vProductAndDescription]([CultureID], [ProductID]);
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Programming

- Example View:

```
-- Postgres
CREATE MATERIALIZED VIEW Production.vProductAndDescription AS
    SELECT  p.ProductID,
            p.Name,
            pm.Name AS ProductModel,
            pmx.CultureID,
            pd.Description,
    FROM    Production.Product p
    INNER JOIN Production.ProductModel pm
        ON  pm.ProductModelID = p.ProductModelID
    INNER JOIN Production.ProductModelProductDescriptionCulture pmx
        ON  pmx.ProductModelID = pm.ProductModelID
    INNER JOIN Production.ProductDescription pd
        ON  pd.ProductDescriptionID = pmx.ProductDescriptionID;

CREATE UNIQUE INDEX IX_vProductAndDescription ON Production.vProductAndDescription(CultureID, ProductID);
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Programming

- Stored Procedures
  - Intended to execute some process, then return
  - Postgres supports multiple languages
    - Trusted: PL/pgSQL, PL/Python, PL/Perl, PL/Tcl
    - Untrusted: PL/v8 (JavaScript), C [Untrusted because they can access outside the database]
  - Does (should) not return data
    - SQL Server side effect of returning data from last SELECT
    - Done before tabular functions were available
- Functions
  - Scalar – Single value return upon each call
    - Used with SELECT or WHERE clause
    - Can get a value from a query result as well
  - Multi-Statement Tabular – Procedural code returns data set
  - Inline Tabular – Single SQL query returns data set



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Programming

- SQL Server Scalar Function:

```
CREATE FUNCTION [dbo].[ufnGetProductDealerPrice](@ProductID [int], @OrderDate [datetime])
RETURNS [money]
AS
-- Returns the dealer price for the product on a specific date.
BEGIN
    DECLARE @DealerPrice money;
    DECLARE @DealerDiscount money;

    SET @DealerDiscount = 0.60  -- 60% of list price

    SELECT @DealerPrice = plph.[ListPrice] * @DealerDiscount
    FROM [Production].[Product] p
        INNER JOIN [Production].[ProductListPriceHistory] plph
        ON p.[ProductID] = plph.[ProductID]
            AND p.[ProductID] = @ProductID
            AND @OrderDate BETWEEN plph.[StartDate] AND COALESCE(plph.[EndDate], CONVERT(datetime, '99991231', 112)); -- Get all the prices!

    RETURN @DealerPrice;
END;
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Programming

- Postgres Scalar Function:

```
CREATE OR REPLACE FUNCTION dbo.ufnGetProductDealerPrice(p_ProductID int, p_OrderDate date)
    RETURNS numeric(20,4)
    STABLE
    LANGUAGE PLPGSQL
AS $function$
DECLARE
    v_DealerPrice numeric(20,4);
    v_DealerDiscount CONSTANT numeric(20,4) := 0.60; -- 60% of list price
-- Returns the dealer price for the product on a specific date.
BEGIN
    SELECT  plph.ListPrice * v_DealerDiscount
    INTO    v_DealerPrice
    FROM    Production.Product p
    INNER JOIN Production.ProductListPriceHistory plph
        ON  plph.ProductID = p.ProductID
    WHERE   p.ProductID = p_ProductID
        AND p_OrderDate BETWEEN plph.StartDate AND COALESCE(plph.EndDate, '9999-12-31'::date); -- Get all the prices!

    RETURN  v_DealerPrice;
END;
$function$;
```

Copyright © 2026, SQLRV, LLC - All Rights Reserved



# Programming

- SQL Server Stored Procedure returning data:

```
01 CREATE PROCEDURE [dbo].[uspGetManagerEmployees]
02    @BusinessEntityID [int]
03 AS BEGIN
04     SET NOCOUNT ON;
05
06     -- Use recursive query to list out all Employees required for a particular Manager
07     WITH [EMP_cte]([BusinessEntityID], [OrganizationNode], [FirstName], [LastName], [RecursionLevel]) -- CTE name and columns
08     AS (
09         SELECT e.[BusinessEntityID], e.[OrganizationNode], p.[FirstName], p.[LastName], 0 -- Get the initial list of Employees for Manager n
10         FROM [HumanResources].[Employee] e
11             INNER JOIN [Person].[Person] p ON p.[BusinessEntityID] = e.[BusinessEntityID]
12         WHERE e.[BusinessEntityID] = @BusinessEntityID
13         UNION ALL
14         SELECT e.[BusinessEntityID], e.[OrganizationNode], p.[FirstName], p.[LastName], [RecursionLevel] + 1 -- Join recursive member to anchor
15         FROM [HumanResources].[Employee] e
16             INNER JOIN [EMP_cte] ON e.[OrganizationNode].GetAncestor(1) = [EMP_cte].[OrganizationNode]
17             INNER JOIN [Person].[Person] p ON p.[BusinessEntityID] = e.[BusinessEntityID]
18         )
19     -- Join back to Employee to return the manager name
20     SELECT [EMP_cte].[RecursionLevel], [EMP_cte].[OrganizationNode].ToString() as [OrganizationNode], p.[FirstName] AS 'ManagerFirstName',
21           p.[LastName] AS 'ManagerLastName', [EMP_cte].[BusinessEntityID], [EMP_cte].[FirstName], [EMP_cte].[LastName] -- Outer select from the CTE
22     FROM [EMP_cte]
23         INNER JOIN [HumanResources].[Employee] e ON [EMP_cte].[OrganizationNode].GetAncestor(1) = e.[OrganizationNode]
24         INNER JOIN [Person].[Person] p ON p.[BusinessEntityID] = e.[BusinessEntityID]
25     ORDER BY [RecursionLevel], [EMP_cte].[OrganizationNode].ToString()
26     OPTION (MAXRECURSION 25)
27 END;
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Programming

- Postgres In-Line Tabular Function returning data:

```
01 CREATE OR REPLACE FUNCTION dbo.uspGetManagerEmployees (p_BusinessEntityID int)
02     RETURNS TABLE ( RecursionLevel smallint,
03                     OrganizationNode text,
04                     ManagerFirstName Name,
05                     ManagerLastName Name,
06                     BusinessEntityID int,
07                     FirstName Name,
08                     LastName Name )
09     STABLE LANGUAGE SQL
10 AS $function$
11     -- Use recursive query to list out all Employees required for a particular Manager
12     WITH RECURSIVE EMP_cte(BusinessEntityID, OrganizationNode, FirstName, LastName, RecursionLevel) AS ( -- CTE name and columns
13         SELECT  e.BusinessEntityID, e.OrganizationNode, p.FirstName, p.LastName, 0::smallint -- Get the initial list of Employees for Manager n
14         FROM    HumanResources.Employee e
15         INNER JOIN Person.Person p ON  p.BusinessEntityID = e.BusinessEntityID
16         WHERE   e.BusinessEntityID = p_BusinessEntityID
17         UNION ALL
18         SELECT  e.BusinessEntityID, e.OrganizationNode, p.FirstName, p.LastName, RecursionLevel + 1::smallint -- Join recursive member to anchor
19         FROM    HumanResources.Employee e
20         INNER JOIN EMP_cte cte ON  cte.OrganizationNode = SUBPATH(e.OrganizationNode, 0, -1) -- Parent Node
21         INNER JOIN Person.Person p ON  p.BusinessEntityID = e.BusinessEntityID
22         WHERE   RecursionLevel < 25
23         )
24     -- Join back to Employee to return the manager name
25     SELECT  cte.RecursionLevel, cte.OrganizationNode::text as OrganizationNode, p.FirstName AS 'ManagerFirstName', p.LastName AS 'ManagerLastName',
26             cte.BusinessEntityID, cte.FirstName, cte.LastName -- Outer select from the CTE
27     FROM    EMP_cte cte
28     INNER JOIN HumanResources.Employee e ON  e.OrganizationNode = SUBPATH(cte.OrganizationNode, 0, -1) -- Parent Node
29     INNER JOIN Person.Person p ON  p.BusinessEntityID = e.BusinessEntityID
30     ORDER BY cte.RecursionLevel, cte.OrganizationNode::text;
31 $function$;
```



# Programming

- Triggers
  - Postgres has two methods
    - Rules
      - Alternative to triggers
      - Modifies or replaces the query before execution
      - May be more efficient than a trigger activated on each row
      - May not be able to handle some complex scenarios
    - Trigger Definition
      - In Postgres, the trigger definition and the action are separate
      - Allows the same action to be performed for multiple triggers
      - Can be performed for each statement or each row
    - Trigger Functions
      - In Postgres, always a function returning a trigger
      - Does not manipulate transactions explicitly



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Programming

- SQL Server Trigger:

```
CREATE TRIGGER [HumanResources].[dEmployee] ON [HumanResources].[Employee]
INSTEAD OF DELETE NOT FOR REPLICATION AS
BEGIN
    DECLARE @Count int;

    SET @Count = @@ROWCOUNT;
    IF @Count = 0
        RETURN;

    SET NOCOUNT ON;

    BEGIN
        RAISERROR
            (N'Employees cannot be deleted. They can only be marked as not current.', -- Message
            10, -- Severity.
            1); -- State.

        -- Rollback any active or uncommittable transactions
        IF @@TRANCOUNT > 0
        BEGIN
            ROLLBACK TRANSACTION;
        END
    END;
END;
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Programming

- Postgres Rule:

```
CREATE OR REPLACE RULE HumanResources.dEmployee AS
    ON DELETE TO TABLE HumanResources.Employee DO INSTEAD
        RAISE NOTICE 'Employees cannot be deleted. They can only be marked as not current.';
```

  - Rule prevents a delete operation from ever occurring
  - No trigger is required as no database operation occurs

- Postgres Trigger Definition:

```
CREATE TRIGGER Purchasing.uPurchaseOrderHeader
    AFTER UPDATE ON Purchasing.PurchaseOrderHeader
    FOR EACH STATEMENT
    EXECUTE FUNCTION Purchasing.uPurchaseOrderHeader_Action();
```



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# Programming

- SQL Server Trigger:

```
CREATE TRIGGER [Purchasing].[uPurchaseOrderHeader] ON [Purchasing].[PurchaseOrderHeader]
AFTER UPDATE AS
BEGIN
    DECLARE @Count int;
    SET @Count = @@ROWCOUNT;
    IF @Count = 0 RETURN;

    SET NOCOUNT ON;
    BEGIN TRY -- Update RevisionNumber for modification of any field EXCEPT the Status.
        IF NOT UPDATE([Status])
        BEGIN
            UPDATE [Purchasing].[PurchaseOrderHeader]
            SET [Purchasing].[PurchaseOrderHeader].[RevisionNumber] = [Purchasing].[PurchaseOrderHeader].[RevisionNumber] + 1
            WHERE [Purchasing].[PurchaseOrderHeader].[PurchaseOrderID] IN (SELECT inserted.[PurchaseOrderID] FROM inserted);
        END;
    END TRY
    BEGIN CATCH
        EXECUTE [dbo].[uspPrintError];

        -- Rollback any active or uncommittable transactions before inserting information in the ErrorLog
        IF @@TRANCOUNT > 0
        BEGIN
            ROLLBACK TRANSACTION;
        END
         EXECUTE [dbo].[uspLogError];
    END CATCH;
END;
```

Copyright © 2026, SQLRV, LLC - All Rights Reserved



# Programming

- Postgres Trigger Function:

```
CREATE OR REPLACE FUNCTION Purchasing.uPurchaseOrderHeader_Action()
    RETURNS TRIGGER
    LANGUAGE PLPGSQL
AS $function$
DECLARE
    v_pg_context text;
    v_pg_sqlstate text;
    v_modified_columns text[];
BEGIN
    IF NOT EXISTS (SELECT 1 FROM NEW HAVING COUNT(1) > 0) THEN
        RETURN;
    END IF;

    SELECT  ARRAY_AGG(LOWER(pre.key)) INTO v_modified_columns
    FROM    jsonb_each(to_jsonb(OLD)) AS pre
    CROSS JOIN jsonb_each(to_jsonb(NEW)) AS post
    WHERE   pre.key = post.key
        AND pre.value IS DISTINCT FROM post.value;

    -- Update RevisionNumber for modification of any field EXCEPT the Status.
    IF NOT ('Status' ILIKE ANY(v_modified_columns)) THEN
        UPDATE  Purchasing.PurchaseOrderHeader poh
            SET RevisionNumber = RevisionNumber + 1
        WHERE   PurchaseOrderID IN (SELECT n.PurchaseOrderID FROM NEW n);
    END IF;
    RETURN NULL; -- result is ignored since this is an AFTER trigger

EXCEPTION
    GET STACKED DIAGNOSTICS v_pg_context = pg_exception_context, v_pg_sqlstate = returned_sqlstate;
    CALL dbo.uspLogError(v_pg_context, v_pg_sqlstate);
END;
$function$;
```

Copyright © 2026, SQLRV, LLC - All Rights Reserved



# Closing Thoughts

- Database migration is not trivial!
  - Proper care and planning are required
  - Review all your database carefully to see what gaps there may be
- This is just a very high-level overview
  - The actual subject is much more complex
  - There are many options in migrating from SQL Server to Postgres
- Many great books on both SQL Server and Postgres
- Not as much written about migrating from SQL Server to Postgres
  - Most focus on data migration only
  - Little about programming aspects
- Reach out with questions!



Copyright © 2026, SQLRV, LLC - All Rights Reserved

# References

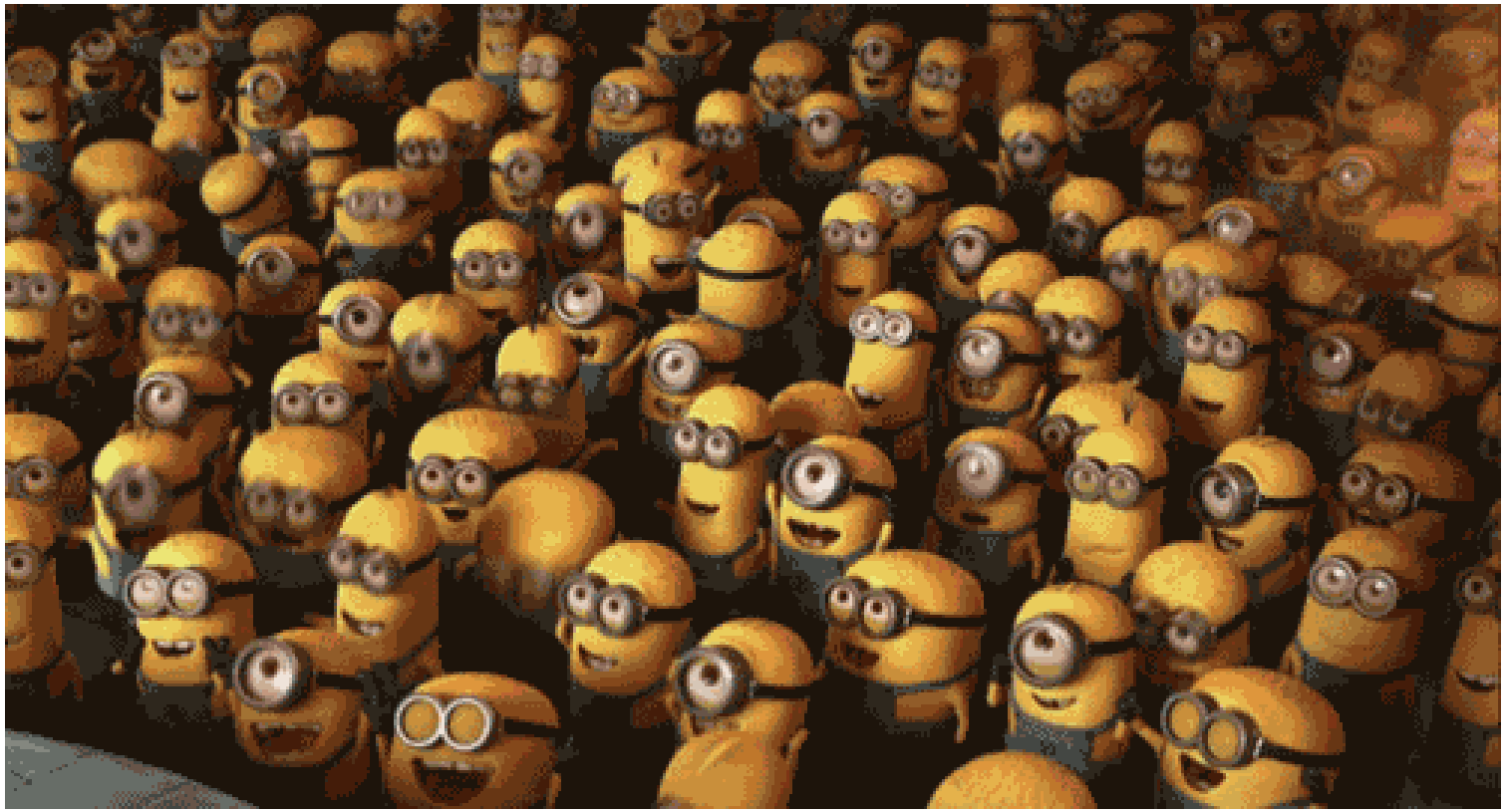
- About Postgres:
  - Introduction to PostgreSQL for the data professional
  - Procedural Programming with PostgreSQL PL/pgSQL
  - PostgreSQL vs SQL Server
  - PostgreSQL History, Features, and SQL Server Comparison
  - SQL Server to PostgreSQL Comparison
- Migrating from SQL Server to Postgres:
  - Migrate Data from SQL Server to PostgreSQL
  - SQL Server to Postgres Database Migration
  - SQL Server to Postgres
  - Migrating Database from SQL Server to PostgreSQL



Copyright © 2026, SQLRV LLC - All Rights Reserved

# Thanks to our Sponsors!

## Without them, we wouldn’t be here!!



Copyright © 2026, SQLRV LLC - All Rights Reserved

# Questions & Comments

**BONUS:**
A **TON** of free eBooks from RedGate and others (databases and programming)!

**PRESENTATION FEEDBACK:**
- Your thoughts needed
- Improve presentations
- Make this event even more valuable!!!


## Aaron N. Cutshall

www.linkedin.com/in/sqlrv
aaron@sqlrv.com
www.sqlrv.com
www.youtube.com/@sqlrv





Copyright © 2026, SQLRV LLC - All Rights Reserved